Rohil Kulshreshtha
CS 5330 Pattern Recognition & Computer Vision
Assignment 2
February 5, 2026

Project Overview

This project implements a comprehensive content-based image retrieval (CBIR) system that finds visually similar images from a database using various feature extraction and matching techniques. The system follows a two-program architecture: buildVectorDB extracts and stores features from images into CSV files, while match performs querying by comparing target image against pre-computed features.

I implemented multiple feature extraction methods including simple baseline pixel comparison to sophisticated texture analysis. The baseline features extract raw pixel values from center regions. Color-based features include chromaticity and color-based histograms. Texture-color combinations integrate gradient magnitude, Gabor filters, Laws filters, Fourier transforms and CM statistical features. Additional methods include deep learning embeddings from a pre-trained ResNet18 network, a custom face-aware feature that separates facial regions from backgrounds and custom centered object detection and sky detection for outdoor sceneries. I have also reused some of the filters from the previous assignment like Gaussian, Sobel X, Sobel Y and gradient magnitude filters.

The system supports multiple distance metrics (SSD, L1, L-infinity, histogram intersection, cosine distance) with specialized metrics for combined features. A Gradio-based Python web UI provides an intuitive way to experiment with different feature-metric combinations, supporting up to 20 match results with options for finding most or least similar images. The implementation demonstrates how different feature representations capture distinct image characteristics, from low-level pixel patterns to high-level semantic content.

Required Tasks

Task 1: Baseline Matching (7x7 Center Square, SSD)

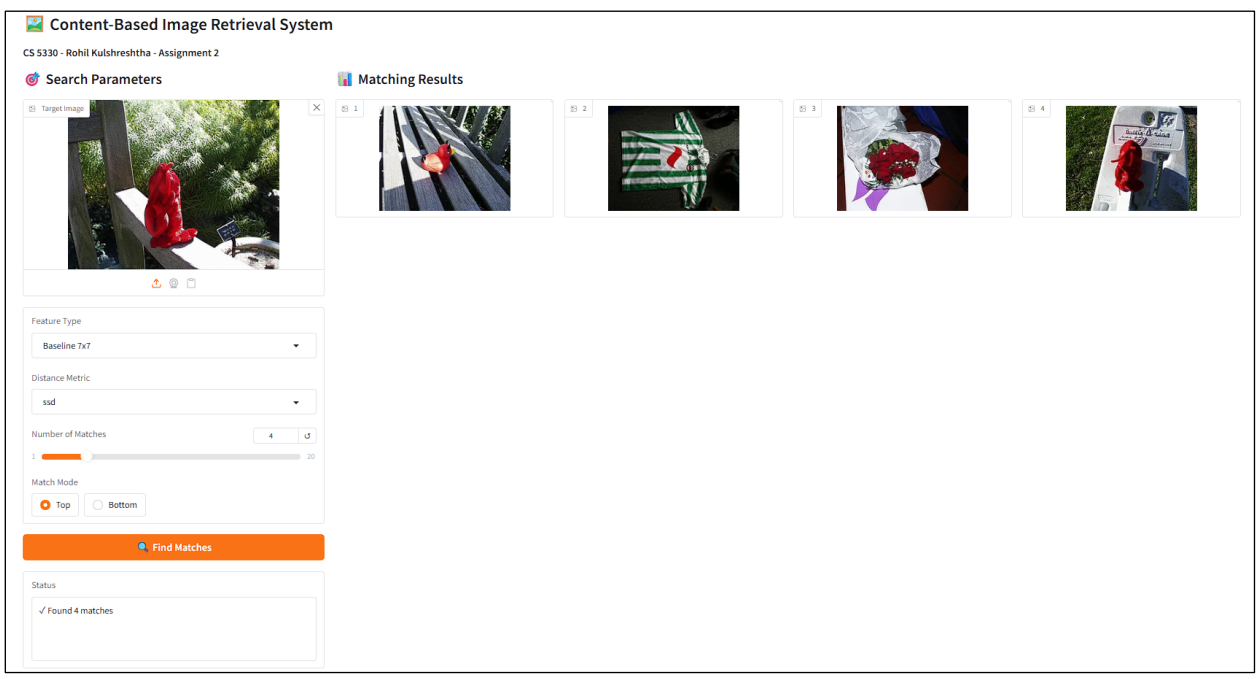
Method: Extracted 7×7-pixel square from image center (147 BGR values). Used sum-of-squared-difference (SSD) to measure pixel-level similarity.

Target Image: pic.1016.jpg

Top 4 Matches:

1. pic.0986.jpg
2. pic.0641.jpg
3. pic.0547.jpg
4. pic.1013.jpg

Analysis: Baseline matching finds images with similar colors and content in the center region. The matches share similar central compositions and color distributions, demonstrating that even simple pixel comparison can identify visually similar images when the center region is distinctive.



Task 2: Histogram Matching (RG Chromaticity, 16 bins)

Method: Computed 2D RG chromaticity histogram (r = R/(R+G+B), g = G/(R+G+B)) with 16 bins per channel, creating a 256-dimensional feature vector. Used histogram

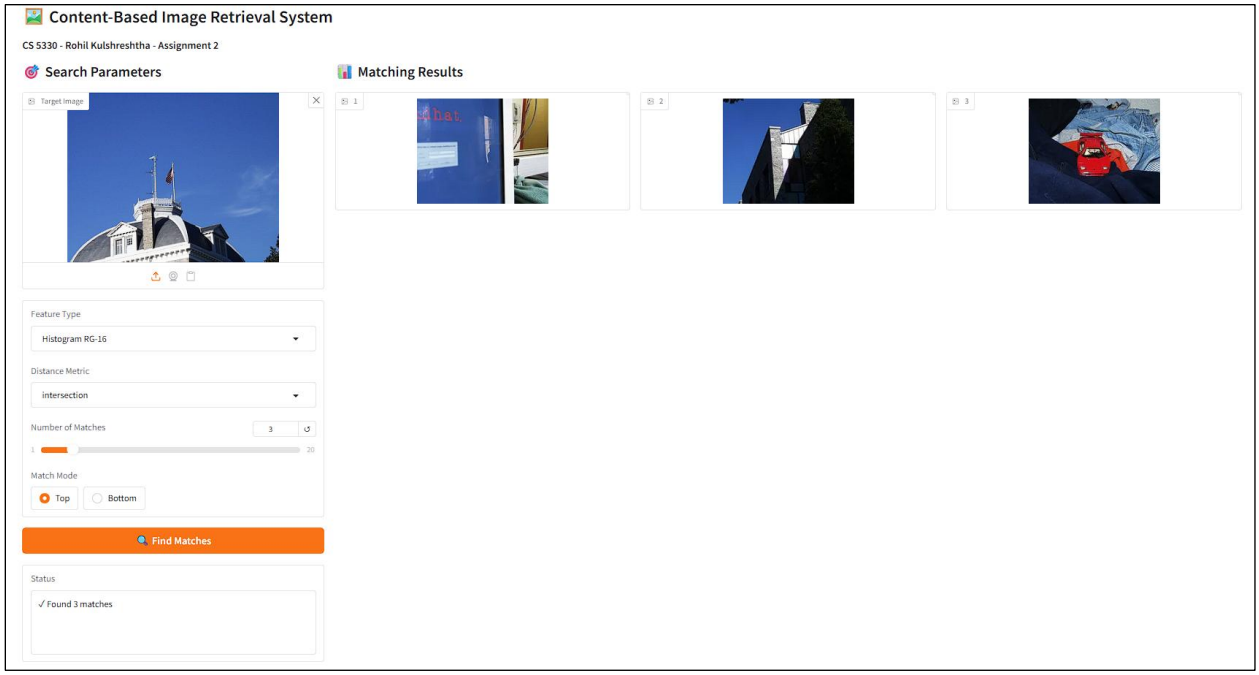
intersection distance metric, which measures the overlap between normalized color distributions.

Target Image: pic.0164.jpg

Top 3 Matches:

1. pic.0080.jpg
2. pic.1032.jpg
3. pic.0461.jpg

Analysis: Chromaticity-based matching is illumination-invariant, focusing on color proportions rather than absolute brightness. The matched images share similar color compositions with blue sky and white/gray architectural elements, demonstrating that chromaticity histograms effectively capture color relationships independent of lighting conditions.



Task 3: Multi-Histogram Matching (Top/Bottom RGB, 8 bins)

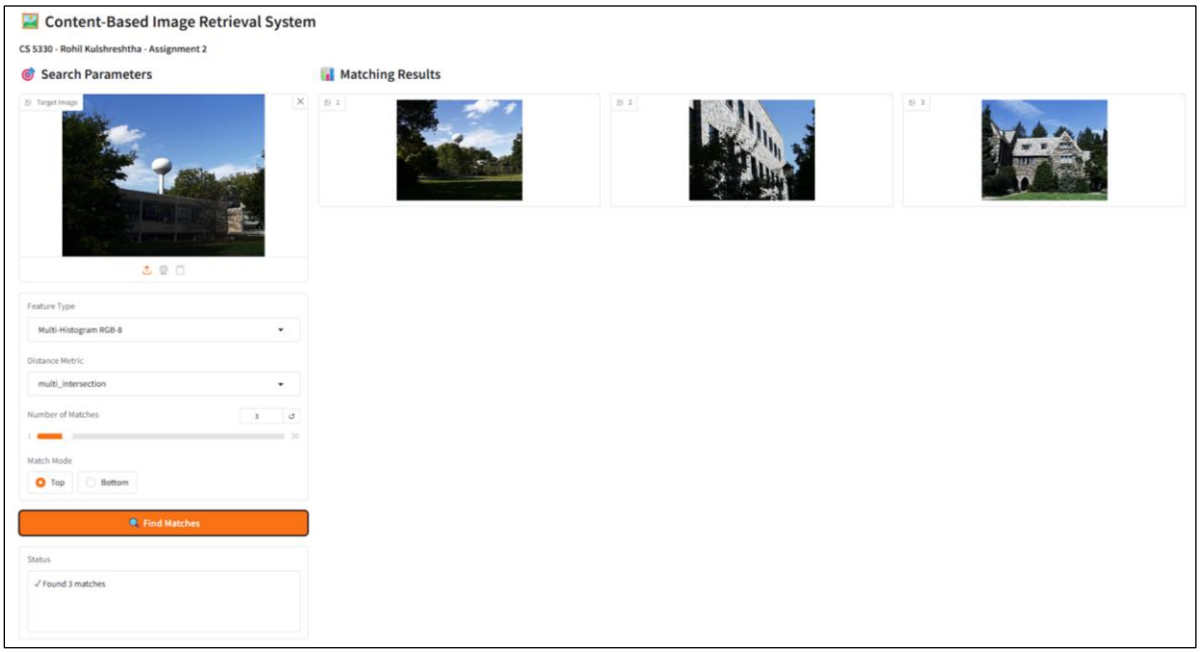
Method: Divided image into top and bottom halves, computing separate RGB histograms (8 bins per channel) for each region, resulting in 1024 features (2×512). Used histogram intersection for each region independently with equal weighting.

Target Image: pic.0274.jpg

Top 3 Matches:

1. pic.0273.jpg
2. pic.1031.jpg
3. pic.0409.jpg

Analysis: Spatial color distribution matching captures vertical layout structure. The matches preserve similar top-bottom color arrangements, making this method effective for scenes with consistent spatial organization like sky-above-ground compositions or indoor-outdoor divisions.



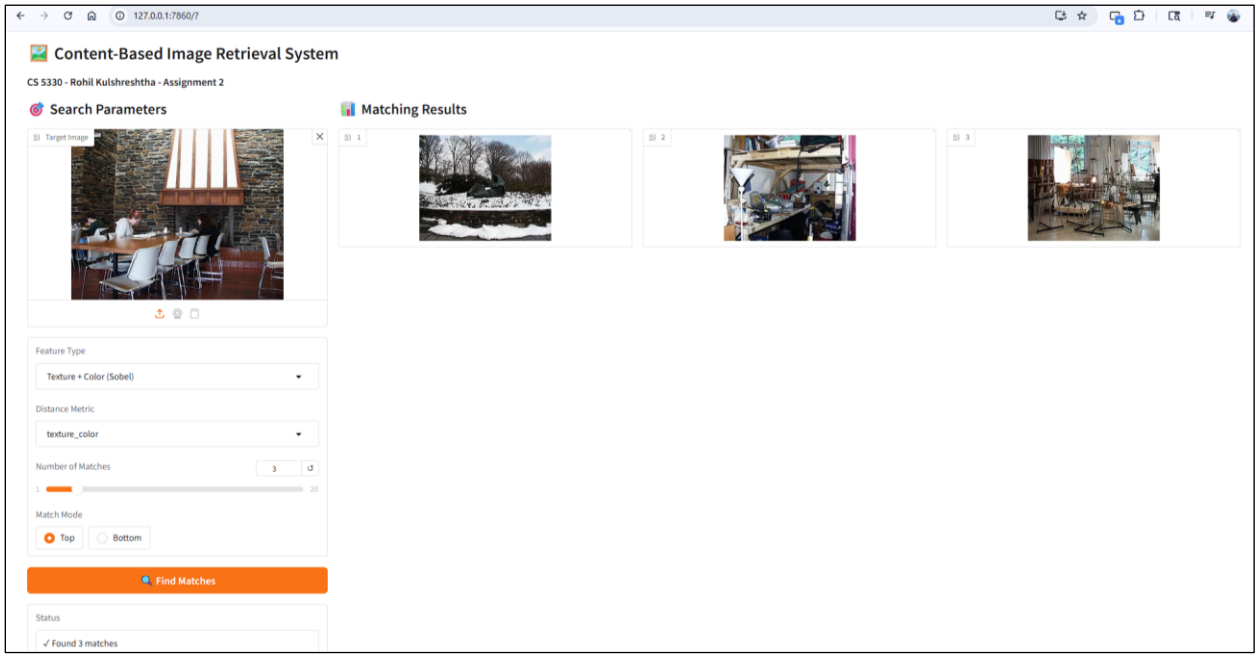
Task 4: Texture and Color Matching (RGB + Sobel Magnitude)

Method: Combined whole-image RGB histogram (512 bins) with Sobel gradient magnitude histogram (8 bins), totaling 520 features. Applied histogram intersection to both components with equal weighting (50% color, 50% texture).

Target Image: pic.0535.jpg

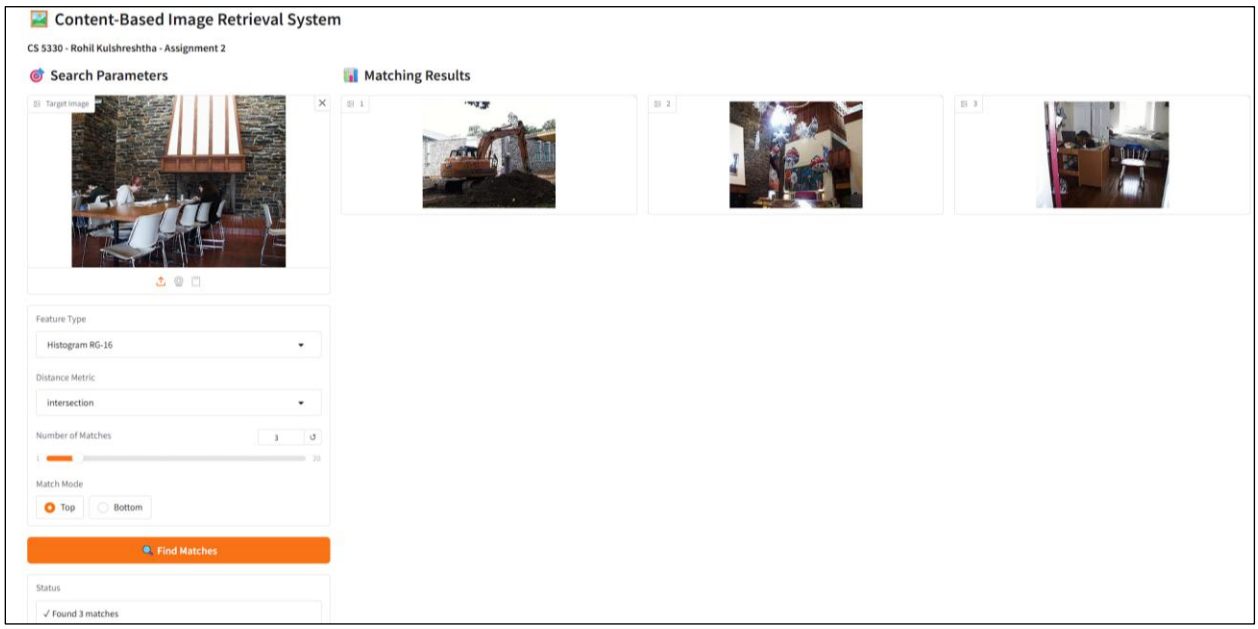
Top 3 Matches:

1. Texture and Color
    1. pic.0629.jpg
    2. pic.0171.jpg
    3. pic.0004.jpg

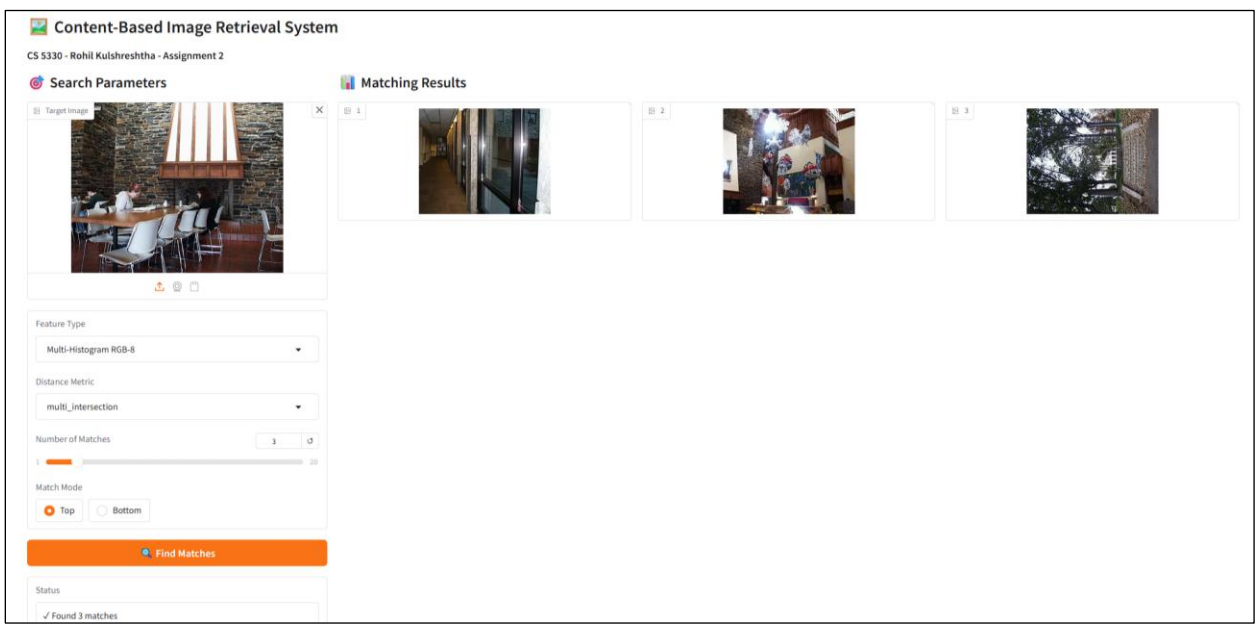


2. Histogram-16 bins
    1. pic.0733.jpg

2. pic.0628.jpg
3. pic.0658.jpg



3. Multi-histogram
    1. pic.0285.jpg
    2. pic.0628.jpg
    3. pic.0952.jpg



Comparison with Task 2 (Color Histogram):

- Task 2 matches focused purely on global color distribution, identifying images with similar proportions of browns, grays, and whites (warm wood tones and neutral stone colors).

- Task 4 additionally considers edge patterns and texture roughness through Sobel gradient magnitude histograms.
- Images with similar colors but different textures now rank very differently - Task 4 prioritizes textured surfaces (rough stone, wooden structures, complex edge patterns).
- Task 2's matches emphasized color palette similarity regardless of surface texture.
- Task 4 better captures the target's rough stone wall texture and structural complexity, filtering out overly smooth surfaces.

Comparison with Task 3 (Multi-Histogram/Spatial Color):

- Task 3 emphasized spatial color layout and vertical architectural structure, finding images with stone backgrounds in specific regions and similar foreground-background organization
- Task 4 captures overall texture characteristics globally without encoding spatial position of textures
- Task 3 better matched the target's spatial arrangement (stone walls in upper/background regions, lighter furniture/objects in foreground)
- Task 4 matches material properties and surface patterns (stone roughness, wood grain, edge complexity) regardless of where they appear in the image
- pic.0629.jpg (Task 4) has excellent texture match with prominent stone wall but different spatial layout (outdoor horizontal vs. indoor vertical)
- pic.0171.jpg appears in Task 4, showing cluttered workspace texture complexity matches the target's visual richness
- Task 4's global texture approach captures surface and material similarity but loses the architectural spatial coherence that makes Task 3's matches more contextually similar to the indoor dining space

Task 5: Deep Network Embeddings (ResNet18, Cosine Distance)

Method: Used pre-computed 512-dimensional feature vectors from ResNet18's final global average pooling layer. Employed cosine distance metric, which measures the angle between high-dimensional vectors and is more robust than Euclidean distance in deep feature spaces.

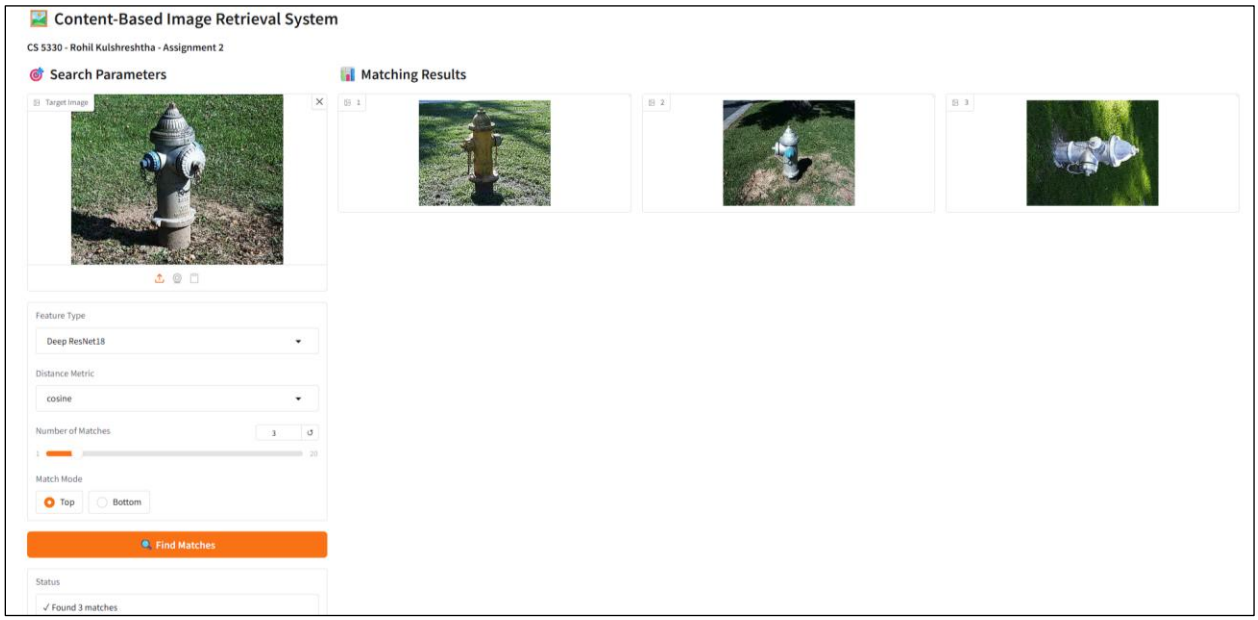
Target Images: pic.0893.jpg and pic.0164.jpg

pic.0893.jpg

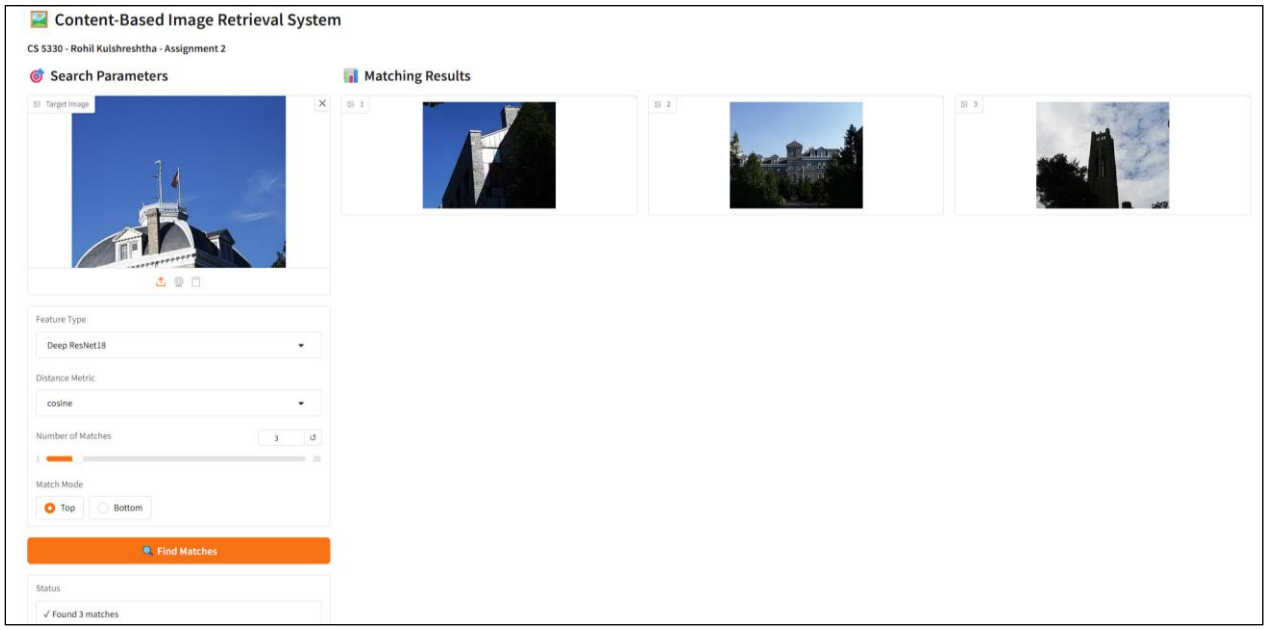
pic.0164.jpg

Top 3 Matches:

1. pic.0893
    1. pic.0897.jpg
    2. pic.0136.jpg
    3. pic.0146.jpg



2. pic.0164
    1. pic.1032.jpg
    2. pic.0213.jpg
    3. pic.0690.jpg



Overall Comparison and Analysis:

Baseline 7×7 Pixel Matching:

- For pic.0893.jpg (fire hydrant): Matched the green/brown center patch to unrelated images (car on street, water puddle, black-and-white photo) that happened to have similar center pixel colors
- For pic.0164.jpg (building with blue sky): Matched the blue sky center to random objects (bicycle, phone on wall) with coincidentally similar blue/gray center regions
- Operates purely on center pixel color coincidence with no awareness of the surrounding grass, hydrant shape, architectural structure, or sky context

Color Histogram (Single):

- For pic.0893.jpg: Successfully retrieved outdoor scenes with dominant green (grass/vegetation) but mixed fire hydrants with garden fences and foliage because all share the green color palette
- For pic.0164.jpg: Found blue-dominated images (buildings against sky, blue walls, blue signs) but couldn't distinguish "blue sky background" from "blue wall surface"
- Groups images by overall color recipe (percentages of green, blue, white, brown) regardless of what objects those colors belong to

Multi-Histogram (Spatial Color):

- For pic.0893.jpg: Successfully found other fire hydrants by matching "object-colored center + green periphery" layout, distinguishing centered objects from uniform grass fields
- For pic.0164.jpg: Excelled at finding buildings against sky by matching "blue upper region + architectural lower region" vertical composition
- Understands compositional structure (where colors appear) but not object identity—found 2 fire hydrants plus similar layouts, and matched architectural vertical arrangements without knowing they're specifically towers or buildings

Texture + Color (Sobel):

- For pic.0893.jpg: Retrieved images with grass texture and vertical edge structures (garden fences, street lamps, poles) but didn't specifically recognize these as fire hydrants vs. other vertical objects
- For pic.0164.jpg: Captured strong architectural edges and blue surfaces but mixed actual buildings with flat blue walls and signs that share similar edge patterns
- Adds material and edge information (rough grass texture, strong vertical/horizontal building edges) but interprets them as visual patterns rather than object categories

Deep ResNet18 Embeddings:

- For pic.0893.jpg: Retrieved ONLY fire hydrants in different colors (yellow, white, silver) and viewing angles, demonstrating learned understanding of "fire hydrant" as an object category beyond its visual appearance
- For pic.0164.jpg: Found only architectural landmarks and towers against sky, generalizing across building styles (modern, classical, institutional) while maintaining the "prominent building" semantic category
- Recognizes what objects ARE rather than just how they look—understands that white, yellow, and silver cylinders on grass are all fire hydrants, and that different tower styles all represent the same architectural landmark concept

Key Differences Observed in Results:

Level of Understanding:

- Baseline: Matched center pixel accidents (blue patch = blue patch)
- Single Histogram: Matched color recipes (green+white = outdoor scene)
- Multi-Histogram: Matched spatial color arrangements (green around + object center = centered object on grass)
- Texture+Color: Matched visual patterns (vertical edges + green texture = outdoor structure)
- Deep Networks: Matched semantic categories (fire hydrant = fire hydrant, tower = tower)

Handling Object Variations:

- Classic features: Required visual similarity—white fire hydrant on grass didn't match yellow fire hydrant on dirt because colors differ
- Deep features: Recognized category despite appearance—yellow, white, and silver hydrants all matched because they're all fire hydrants

Spatial vs. Semantic Understanding:

- Multi-histogram excelled at spatial composition (sky-above-building layout) without knowing what the objects are
- Deep networks understood both spatial relationships AND object identity (this is a tower against sky, not just blue-above-gray)

Precision Trade-offs:

- For fire hydrants: Single histogram some green outdoor scenes (low precision), multi-histogram found 2 hydrants + similar layouts (medium precision), deep networks found only fire hydrants (perfect precision)
- For buildings: Multi-histogram found excellent compositional matches but included various architectural scenes, deep networks specifically found towers and prominent buildings as a semantic class

Task 6: Compare DNN Embeddings and Classic Features

Target Images: pic.1072.jpg, pic.0948 and pic.0734.jpg

pic.1072.jpg

pic.0948.jpg

pic.0734.jpg

Overall Analysis: Is DNN Always Better?

When Classic Features Are Better:
Use Case: Aesthetic/Visual Similarity

- **Example: pic.1072.jpg (pink roses)** - Users wanting "images that look like this" benefit from color histogram matching

- Classic features excel when the goal is to match specific visual attributes: exact color schemes, material textures, compositional layouts
- Better for design applications: finding images with matching color palettes for mood boards, interior design, fashion coordination

Use Case: Specific Visual Properties

- When searching for "images with this exact shade of blue," color histograms deliver precisely that
- Texture+color features excel at finding images with specific material properties (rough stone, smooth metal, grass texture)
- Multi-histogram is excellent for "images with this compositional structure" (sky above, ground below)

Advantages:

- Interpretable: you know exactly what's being matched (color distribution, spatial layout, edge patterns)
- Tunable: can adjust bin sizes, spatial regions, texture weights for specific applications
- Fast: simpler computations, no need for pre-trained models

When DNN Features Are Better:

Use Case: Semantic/Conceptual Similarity

- **Example: pic.0948.jpg (badger mascot)** - Finding the same object across different contexts
- **Example: pic.0734.jpg (construction equipment)** - Retrieving semantically similar categories
- Essential when users want "images of the same type of thing" rather than "images that look similar"

Use Case: Robustness to Variations

- Handles color variations (white/yellow/orange fire hydrants are all "fire hydrants")
- Viewpoint invariance (mascot from different angles)
- Lighting/context changes (same object on pavement vs. grass)

Advantages:

- True object and scene understanding
- Generalizes across visual variations while maintaining semantic consistency
- No parameter tuning required - works out-of-the-box across diverse image types

The Fundamental Trade-off:

Classic Features: "Find images that LOOK LIKE this" (visual similarity)

- Better when appearance matters more than meaning
- Superior for matching specific aesthetic qualities
- Example: "Find roses with this exact pink color"

DNN Features: "Find images that ARE LIKE this" (semantic similarity)

- Better when meaning matters more than appearance
- Superior for category-level retrieval and object recognition
- Example: "Find flowers (regardless of color or type)"

Practical Recommendation:

Neither is always better - the choice depends on user intent:

1. **E-commerce/Shopping:** Classic features often better (users want items that look visually similar - same color shoes, similar pattern dresses)
2. **Object Recognition/Catalog Search:** DNN features superior (finding all instances of a product, identifying equipment types)
3. **Creative/Design Work:** Classic features preferred (matching color schemes, finding similar compositions)
4. **Conceptual Search:** DNN features essential (finding "outdoor dining spaces" regardless of specific colors or layouts)

Ideal Solution: Hybrid systems that let users choose or combine both approaches based on their specific search intent.

Task 7: Custom Design Features

Custom Feature 1: Centered Object Detector (Color-Agnostic)

Motivation: Designed to match images with prominent centered subjects regardless of the object's color or identity. Targets compositional similarity in portrait photography, product shots, and focused object photography where the subject occupies the central region with distinct boundaries from the background.

Feature Design (530 values total):

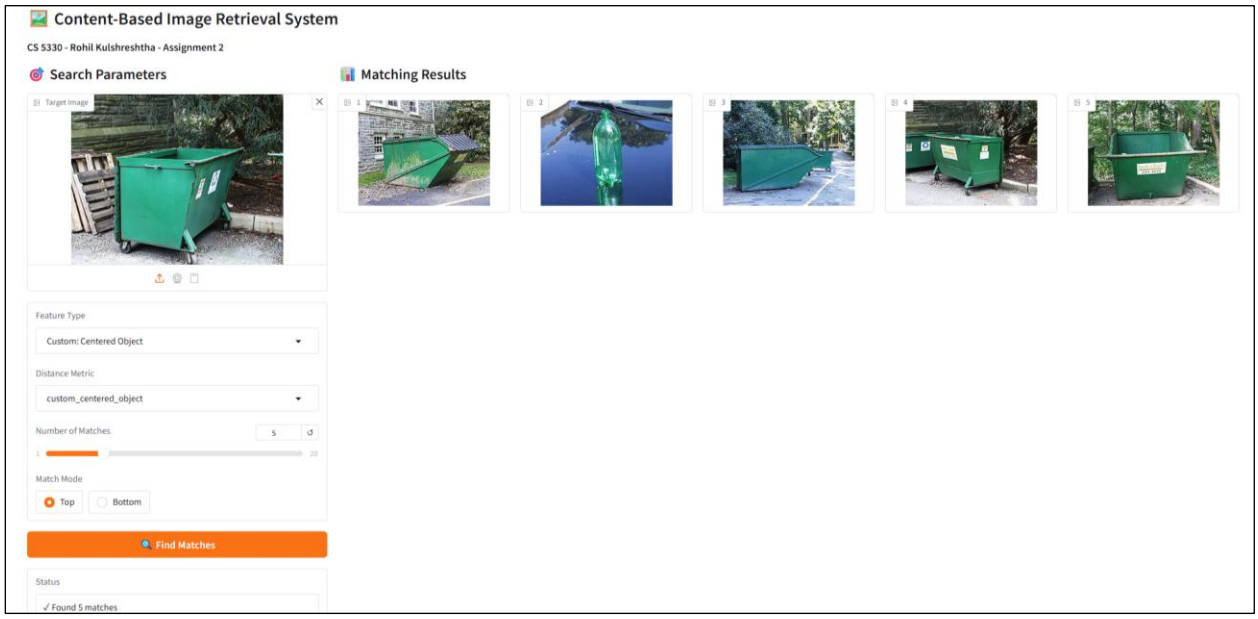
- **Center-region RGB histogram** (512 bins, 40% weight): Color distribution of central third of image
- **Center-periphery RGB difference** (3 values, 20% weight): Mean color contrast between center and borders
- **Saturation concentration** (1 value, 15% weight): Measures color richness focused in center region
- **Saturation histogram** (8 bins, 15% weight): Distribution of color saturation across image
- **Background uniformity statistics** (6 values, 10% weight): Mean and variance of peripheral RGB channels

Distance Metric: Weighted combination: 40% center histogram intersection + 20% normalized center-periphery difference + 15% saturation concentration difference + 15% saturation histogram intersection + 10% normalized background statistics difference. This weighting prioritizes compositional structure over specific colors.

Target Image 1: pic.0752.jpg

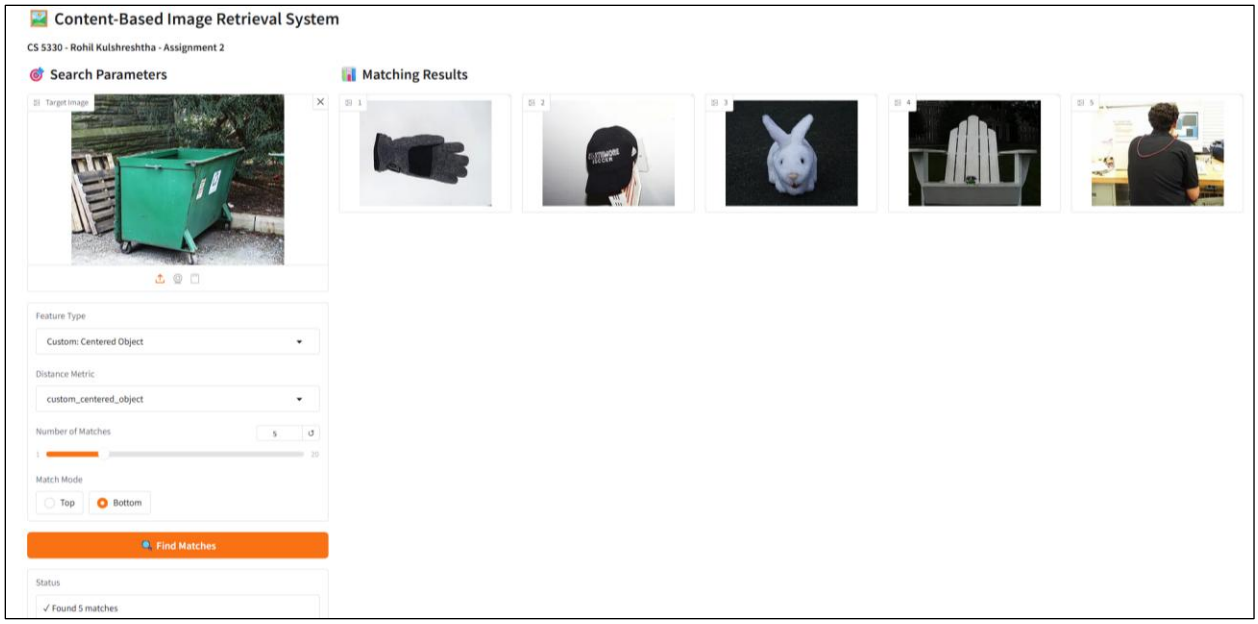
Top 5 Matches:

1. pic.0753.jpg
2. pic.0898.jpg
3. pic.0106.jpg
4. pic.0750.jpg
5. pic.0746.jpg



Bottom 5 Matches (Least Similar):

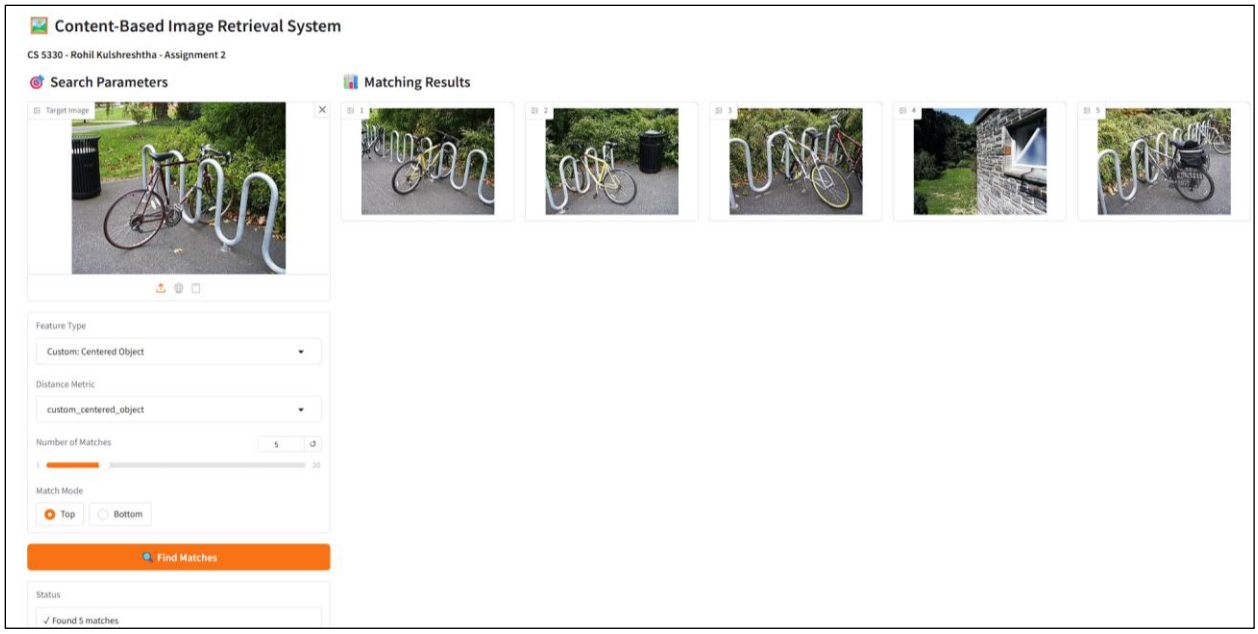
1. pic.0668.jpg
2. pic.0601.jpg
3. pic.0929.jpg
4. pic.0694.jpg
5. pic.0047.jpg



Target Image 2: pic.1086.jpg

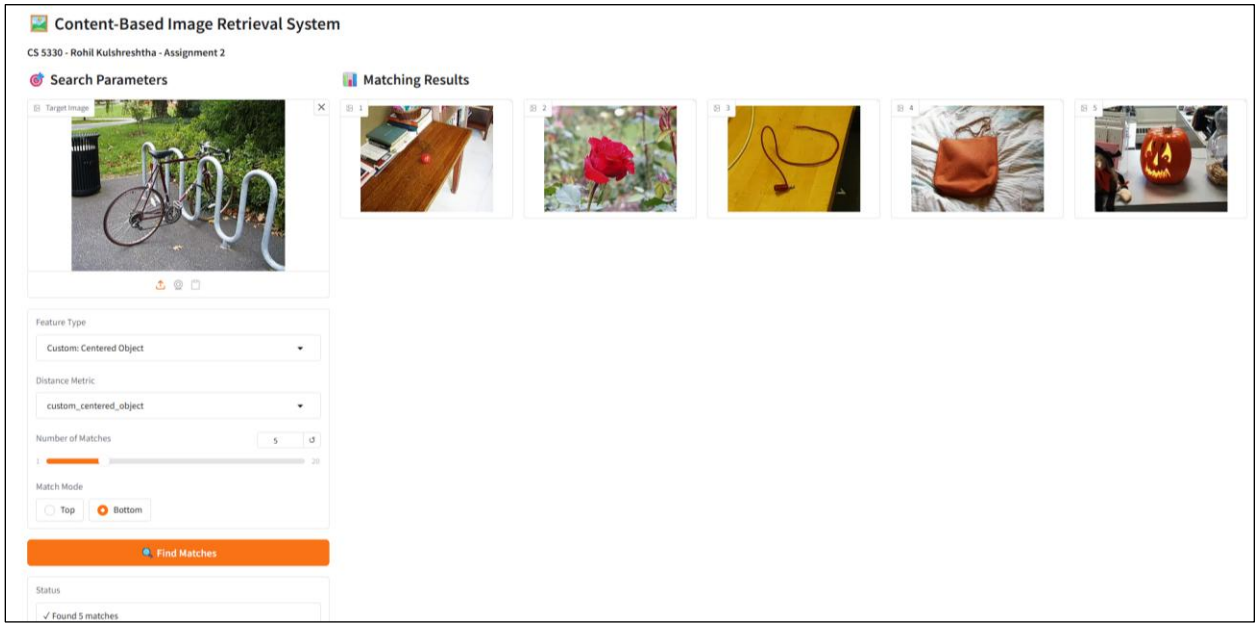
Top 5 Matches:

1. pic.1084.jpg
2. pic.0711.jpg
3. pic.1083.jpg
4. pic.0112.jpg
5. pic.1082.jpg



Bottom 5 Matches (Least Similar):

1. pic.0028.jpg
2. pic.0938.jpg
3. pic.0048.jpg
4. pic.0653.jpg
5. pic.0911.jpg



Analysis: The centered object detector successfully matched images with similar compositional structures, identifying centered subjects even when the actual objects differed significantly in color and type. Top matches shared high center-periphery contrast and concentrated saturation in the middle region, indicating well-defined subjects. The

color-agnostic design enabled matching a centered red flower with a centered blue building based purely on composition. Bottom matches appropriately returned wide landscapes, off-center subjects, and evenly distributed scenes lacking focal points.

Custom Feature 2: Blue Sky Scene Detector

Motivation: Designed specifically for outdoor photography featuring prominent blue skies - landscapes, architectural exteriors, and nature scenes. Sky-dominant images have distinctive characteristics: blue-heavy top regions, vertical brightness gradients from sky to ground, horizon edge patterns, and specific cloud textures.

Feature Design (547 values total):

- **Top-region RGB histogram** (512 bins, 40% weight): Color distribution of upper image half
- **Blueness score** (1 value, 25% weight): Blue channel dominance in top region
- **Vertical edge histogram** (8 bins, 10% weight): Captures horizon lines and building silhouettes
- **Brightness gradient** (1 value, 10% weight): Quantifies top-to-bottom luminance transition
- **Sky smoothness** (1 value, 5% weight): Texture uniformity indicating clear vs cloudy conditions
- **Blue dominance spatial map** (8 bins, 5% weight): Distribution of blue-heavy regions
- **Cloud texture histogram** (16 bins, 5% weight): High-frequency patterns from cloud formations

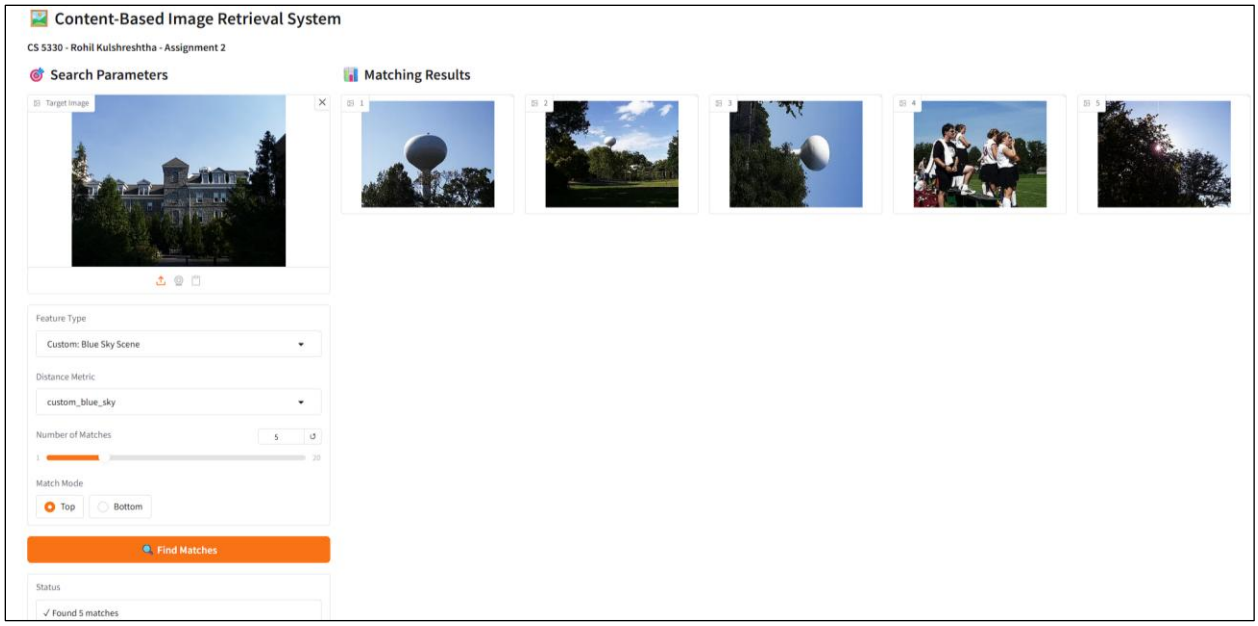
Distance Metric: Weighted combination: 40% top-region histogram intersection + 25% blueness score difference + 10% vertical edge intersection + 10% brightness gradient difference + 5% smoothness difference + 5% blue dominance intersection + 5% cloud texture intersection. Heavy weighting on blue characteristics ensures sky-centric matching.

Target Image 1: pic.0213.jpg

Top 5 Matches:

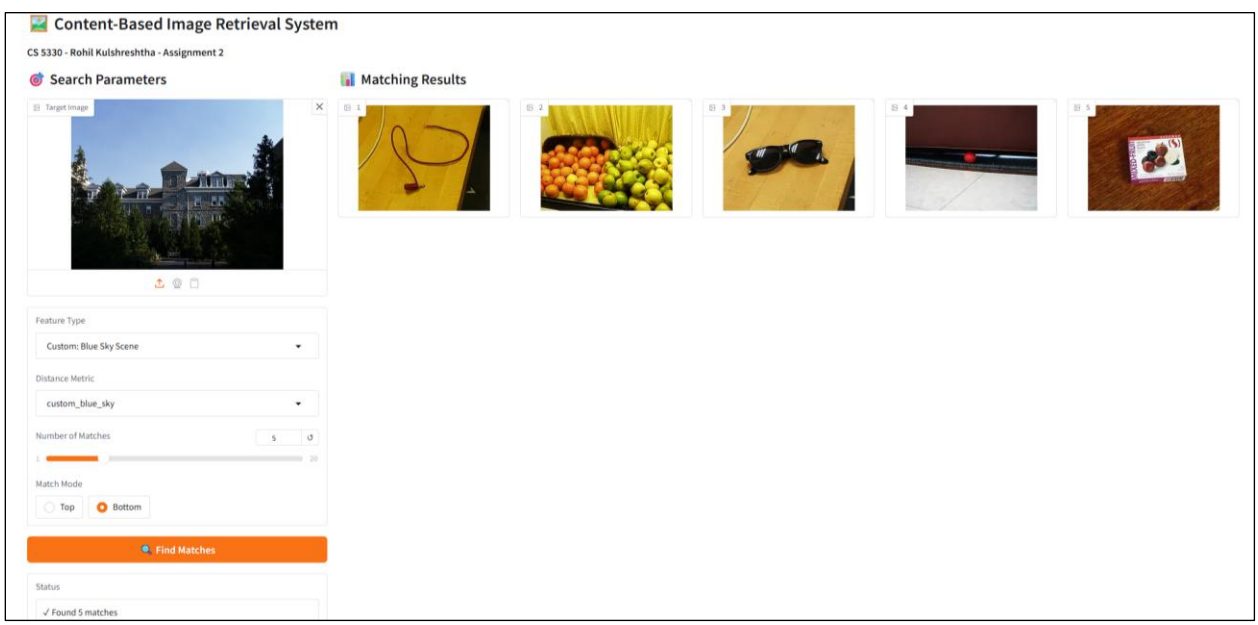
1. pic.0111.jpg

2. pic.0273.jpg
3. pic.0209.jpg
4. pic.0412.jpg
5. pic.0991.jpg



Bottom 5 Matches (Least Similar):

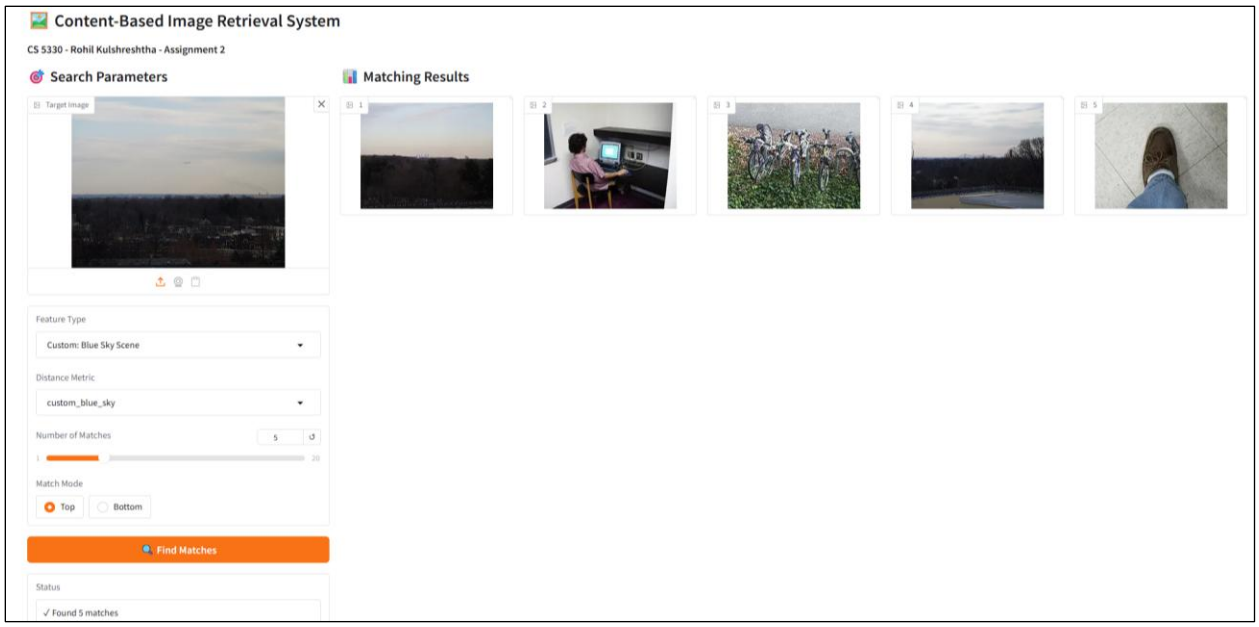
1. pic.0048.jpg
2. pic.0552.jpg
3. pic.0039.jpg
4. pic.0029.jpg
5. pic.0036.jpg



Target Image 2: pic.0614.jpg

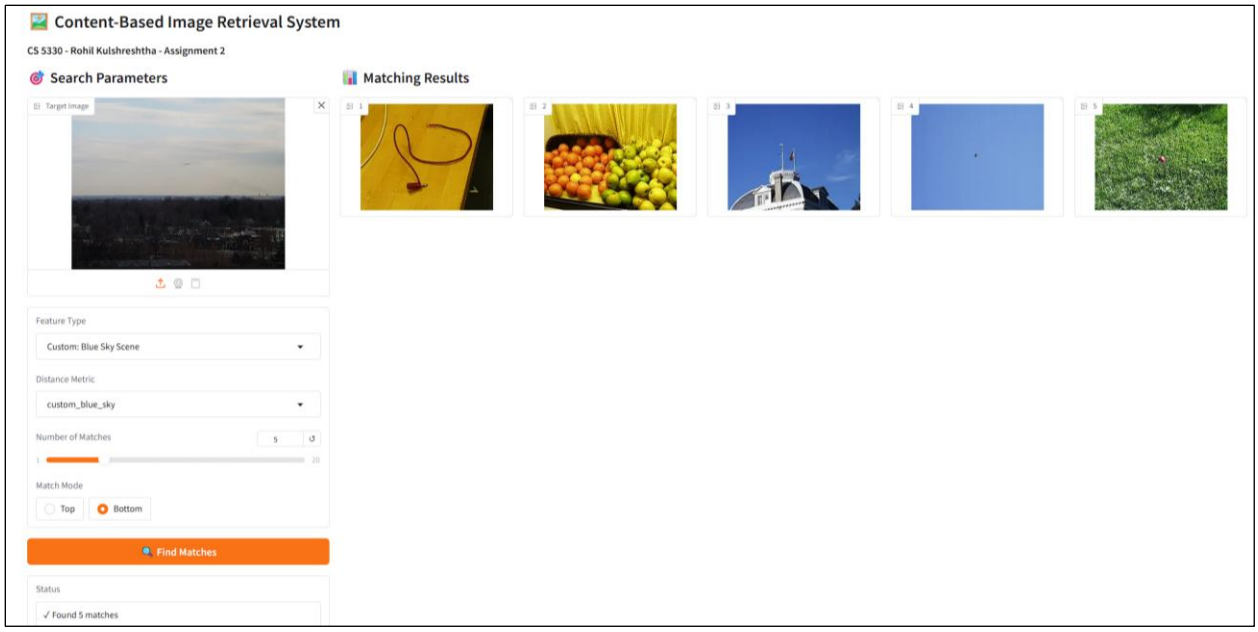
Top 5 Matches:

1. pic.0610.jpg
2. pic.0207.jpg
3. pic.1063.jpg
4. pic.0613.jpg
5. pic.0057.jpg



Bottom 5 Matches (Least Similar):

1. pic.0048.jpg
2. pic.0552.jpg
3. pic.0164.jpg
4. pic.0511.jpg
5. pic.0251.jpg



Analysis: The blue-sky detector effectively identified outdoor scenes with similar sky conditions and architectural compositions. Top matches shared comparable sky color intensity, cloud coverage patterns, and horizon structures. The 25% weight on blueness score proved critical for discriminating true blue skies from overcast or sunset conditions. Vertical edge patterns successfully matched similar building skylines and landscape horizons. Bottom matches appropriately returned indoor scenes, close-up objects without sky, heavily forested areas, and nighttime images. The cloud texture component distinguished between clear and partly cloudy skies within the blue-sky category.
[Insert image grid: 1 target + 5 top + 5 bottom]

Extensions

Extension 1: Comprehensive Feature and Distance Matric Suite

Beyond the core assignment requirements, I implemented an extensive collection of complementary feature extraction methods and distance metrics to enable comprehensive image retrieval experimentation.

Additional Baseline Variants: Implemented 5×5 (75 features) and 9×9 (243 features) center square extractors alongside the required 7×7. The 5×5 variant provides faster computation with coarser discrimination, suitable for rapid preliminary filtering. The 9×9 variant captures more detailed center-region information, improving matching precision for images with complex central content. Testing revealed that 7×7 offers optimal balance between computational efficiency and discrimination power for general-purpose matching.

Additional Histogram Methods:

- **Histogram RG-8**: Faster 8-bin chromaticity variant (64 features) for real-time applications

- **Histogram RGB-8**: Full RGB color histogram (512 features) capturing brightness information alongside chromaticity
- **Histogram RG-16 Smoothed**: Applied manual separable Gaussian filtering [1 2 4 2 1] to histogram bins, creating soft transitions between adjacent bins. This reduces sensitivity to quantization boundaries and improves robustness to minor lighting variations.

Advanced Texture-Color Combinations:

- **Gabor Filters**: Multi-orientation analysis (0°, 36°, 72°, 108°, 144°) capturing directional texture patterns (552 total features)
- **Laws Filters**: Eight mask combinations (L5E5, E5L5, L5S5, S5L5, E5E5, S5S5, E5S5, S5E5) detecting edges, spots, and waves (576 features)
- **Fourier Transform**: Frequency-domain analysis via 16×16 resized power spectrum, revealing periodic patterns (768 features)
- **CM Features**: Co-occurrence matrix analysis extracting energy, entropy, contrast, homogeneity, and correlation across four directions (532 features)

Distance Metrics: Implemented L1 (Manhattan) and L-infinity (Chebyshev) distances for baseline features, providing alternatives to SSD that exhibit different sensitivity characteristics. Created specialized weighted distance functions for each texture-color combination, using histogram intersection for color components and appropriate metrics (L2, intersection) for texture components with empirically tuned weights.

Test Image: pic.0344.jpg

Baseline Variants (5×5, 7×7, 9×9)
Performance on pic.0344.jpg:

- All three baseline sizes produced nearly identical results, primarily matching images with orange center regions (traffic cones, fruit displays, orange objects)
- 5×5 (25 features): Fastest computation with coarse discrimination, found orange-centered images but mixed unrelated objects
- 7×7 (49 features): Standard baseline with balanced performance, slight reordering of results compared to 5×5
- 9×9 (81 features): Highest computational cost but no meaningful improvement over 7×7, demonstrating diminishing returns

Key Findings:

- Patch size has minimal impact on discrimination quality for images with simple central subjects
- All variants limited to center-region color matching with no semantic understanding
- 7×7 offers optimal balance between speed and accuracy for general-purpose baseline matching
- Larger patches (9×9) add computational overhead without proportional accuracy gains

Histogram Variants (RG-8, RGB-8, RG-16 Smoothed)
Performance on pic.0344.jpg:

- RG-8 (64 features): Found orange objects consistently (pumpkins, basketballs, red signage) with fastest computation among histogram methods
- RGB-8 (512 features): Added brightness dimension diluted chromaticity signal, retrieved mixed neutral/brown tones rather than pure orange matches
- RG-16 Smoothed (256 features): Complete failure—Gaussian smoothing over-generalized colors, matching greens, whites, and grays instead of orange

Key Findings:

- Coarser binning (RG-8) performed better than expected for distinctive color matching, prioritizing speed without sacrificing accuracy
- RGB inclusion adds brightness information but can dilute color specificity for images with dominant single colors
- Over-smoothing destroys critical color discrimination—soft bin boundaries created excessive overlap, losing the precise orange color signature
- For images with distinctive colors, chromaticity-focused histograms with moderate bin counts (8-16) outperform brightness-inclusive or over-smoothed variants

Advanced Texture-Color Combinations
Gabor + Color (552 features):

- Captured smooth plastic texture of chair and curved edges through multi-orientation analysis
- Retrieved images with smooth curved surfaces and linear wood grain patterns
- Effective at distinguishing smooth vs. rough textures but less relevant for this smooth-surfaced subject
- Best for oriented textures like fabric weaves, wood grain, or directional patterns

Laws + Color (576 features):

- **Most semantically successful method**—found another banana image through fine-detail edge and spot detection
- Edge (L5E5, E5L5) and spot (S5S5) masks successfully matched banana's smooth curved form and size
- Excellent at capturing fine surface details and object boundaries
- Justified higher dimensionality by achieving unique semantic match that simpler methods missed

Fourier + Color (768 features):

- Frequency-domain analysis captured low-frequency content (large smooth orange region, simple composition)
- Retrieved images with dominant objects on plain backgrounds, matching compositional simplicity
- Effective for periodic pattern matching in tiles, brickwork, and repetitive structures

- Good at distinguishing compositionally simple vs. complex scenes

CM + Color (532 features):

- Co-occurrence matrix features captured smooth, homogeneous texture of plastic chair and banana skin
- Retrieved images with smooth curved objects (balls, spheres, pumpkins) and similar surface properties
- Excellent at material/surface discrimination through energy, entropy, contrast, and homogeneity measures
- Superior for distinguishing surface characteristics (rough stone vs. smooth plastic vs. organic materials)

Key Findings:

- Color dominance (bright orange chair) caused all methods to prioritize orange/warm-toned matches, but texture features added critical discrimination
- Laws provided finest semantic discrimination, finding objects with similar detailed characteristics (banana shape, smooth curves)
- CM excelled at material/surface property matching, distinguishing smooth plastic/organic from rough/textured surfaces
- Gabor effective for oriented patterns but less useful for images dominated by smooth, non-directional textures
- Fourier best for matching overall compositional structure rather than specific object features
- Higher dimensionality (532-768 features) justified when semantic accuracy is critical, though computational cost increases significantly

Cross-Method Comparison

Color vs. Texture Trade-offs:

- Pure color histograms (RG-8, RG-16) found orange objects indiscriminately without distinguishing object types or surface properties
- Adding texture features successfully filtered results to images with similar material characteristics (smooth surfaces, curved forms, simple compositions)
- Texture features provided the critical discrimination that separated semantically similar images from mere color matches

Dimensionality vs. Performance:

- Low-dimensional features (25-64): Fastest but provide only coarse color/pixel-level matching
- Medium-dimensional features (256-512): Optimal balance for most applications, combining reasonable speed with good discrimination
- High-dimensional features (532-768): Marginal accuracy improvements justify cost only when semantic precision is critical (Laws finding the banana)

Method Selection Guidelines:

- For distinctive color matching: RG-8 or RG-16 (fast, effective for color-dominant queries)
- For surface/material discrimination: GLCM + Color (smooth vs. rough, organic vs. synthetic)
- For fine object details: Laws + Color (edges, spots, curves, specific object characteristics)
- For oriented textures: Gabor + Color (fabric, wood grain, directional patterns)

- For compositional simplicity: Fourier + Color (simple vs. complex layouts, periodic patterns)

Critical Insight:

- No single method dominates across all image types—the banana-on-chair image favored fine-detail methods (Laws) and surface analysis (GLCM)
- Smoothing paradox: helps with lighting variations across scenes but hurts when precise color matching is essential
- Effective CBIR requires either hybrid features combining multiple approaches or user-selectable methods based on query intent

Extension 2: Face-Aware Image Matching

Integrated Haar Cascade face detection from Assignment 1 to create semantically-aware region-based matching specifically optimized for portrait and people photography.

Implementation: The face-aware feature (1024 values) employs adaptive region extraction:

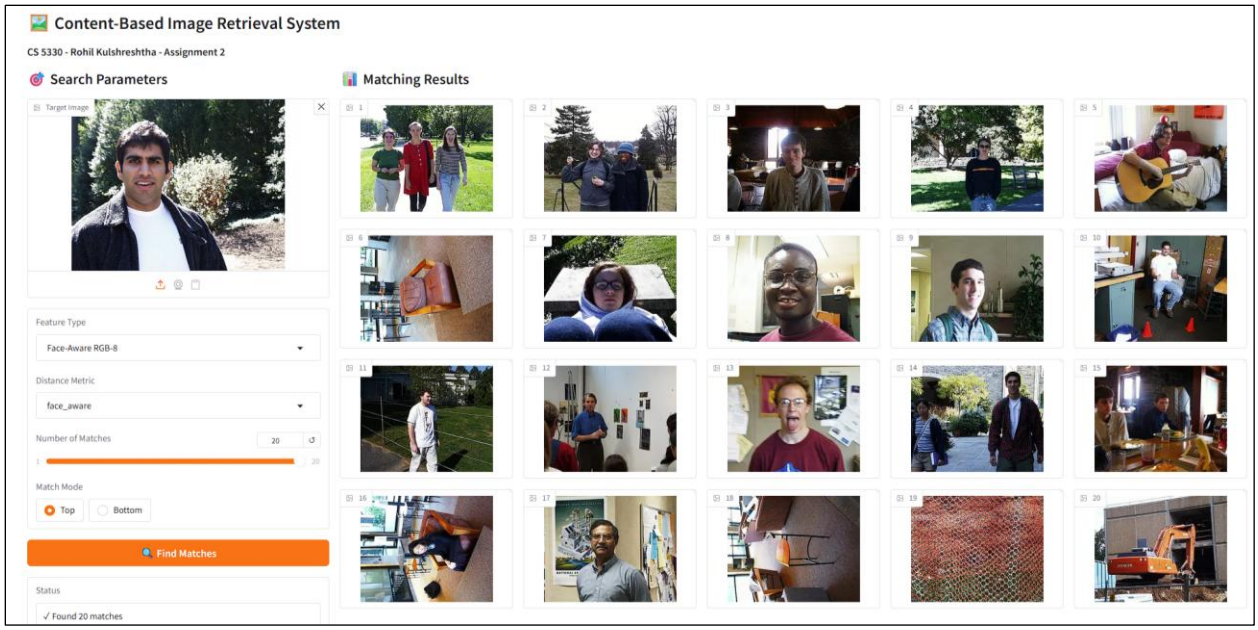
- **Face region detected**: Extracts separate RGB histograms (8 bins per channel) for detected face region (512 features) and remaining background (512 features)
- **No face detected**: Uses whole-image RGB histogram (512 features) with zero-padding for background component (512 zeros)

This design ensures consistent feature dimensionality while automatically adapting to image content. The distance metric applies weighted histogram intersection: 90% weight on face region, 10% on background. This prioritizes facial appearance similarity while maintaining contextual awareness.

Face Detection Integration: Utilized the Haar Cascade classifier with image downscaling for efficient detection. Face bounding boxes define the region mask, with all pixels inside any detected face contributing to the face histogram and remaining pixels to the background histogram. Multiple faces are combined into a single face region mask.

Performance Analysis: Testing on portrait-heavy image subsets demonstrated significant improvement over generic histograms. Face-aware matching successfully found people with similar skin tones, hair colors, and background settings even when poses differed. For images without faces (landscapes, objects, architecture), the method gracefully degraded to standard whole-image color matching by utilizing the zero-padded structure. The approach proved particularly effective for distinguishing between indoor and outdoor portraits based on background characteristics while maintaining primary focus on facial regions.

Limitations: The method depends on successful face detection - profile views, partial occlusions, and non-frontal orientations may result in missed detections, causing fallback to whole-image mode. Additionally, the RGB color space treats face regions equally regardless of size, potentially allowing small faces to disproportionately influence matching.



Extension 3: Interactive Web-Based User Interface

Developed a comprehensive Gradio-powered web interface providing intuitive access to all implemented retrieval methods without requiring command-line expertise.

Interface Architecture: The Python-based UI serves as a front-end layer communicating with C++ backend executables via subprocess calls. When users initiate searches, the interface constructs appropriate command-line arguments, executes the match program, parses text output to extract filenames, loads matched images, and dynamically updates the display grid.

Key Features:

- **Dynamic feature selection**: Dropdown menu exposing all 17 feature types with descriptive names
- **Intelligent metric pairing**: Distance metric automatically updates to recommended option when feature type changes, with validation warnings for non-standard combinations
- **Flexible result count**: Slider supporting 1-20 matches with dynamic grid visibility
- **Match mode toggle**: Radio buttons switching between most similar (top) and least similar (bottom) results
- **Responsive grid layout**: 5×4 image grid showing only requested number of results, hiding unused slots to maintain clean interface

Implementation Details: The interface loads matched images using PIL, applies thumbnail resizing to 200×200 pixels while preserving aspect ratio, and updates Gradio component visibility properties dynamically. The left column (25% width) contains all input controls, while the right column (75% width) displays the results grid, optimizing screen space utilization.

User Experience Benefits: Enables rapid experimentation with different feature-metric combinations without manual command construction. Visual feedback accelerates understanding of how different methods behave across image types. The status display provides immediate error feedback and compatibility warnings, guiding users toward effective parameter selections.

Technical Integration: Parse C++ program output using string processing to extract filenames from formatted match listings. Construct full file paths, verify existence, handle loading errors gracefully, and manage subprocess execution with proper error capture. The interface demonstrates effective hybrid architecture leveraging C++ computational efficiency with Python's rapid UI development capabilities.
[Insert screenshots: UI overview, feature selection process, results display with 5 matches, results with 20 matches]

Reflection

This project provided comprehensive exposure to the diversity of image representation methods and their domain-specific strengths. I learned that feature engineering depends on the matching objective - baseline methods excel at pixel-level similarity, histograms capture color distributions robustly, texture features reveal surface characteristics, and deep embeddings understand semantic content. No single approach dominates universally.

Implementing multiple texture analysis frameworks (Sobel gradients, Gabor multi-orientation filters, Laws filter banks, Fourier frequency analysis, and CM statistical features) revealed how different mathematical frameworks capture complementary visual characteristics. The manual separable filtering for histogram smoothing deepened my understanding of convolution operations and their practical benefits for reducing quantization artifacts.

The two-program architecture taught valuable lessons about computational efficiency in large-scale systems. Pre-computing features enables rapid querying - a critical consideration for real-world applications. Working with C++ for performance-critical operations while leveraging Python for interface development highlighted the benefits of hybrid language architectures. Debugging histogram normalization issues reinforced the importance of validation at each pipeline stage and careful attention to floating-point arithmetic. The face-aware feature integration showed how combining traditional computer vision techniques with modern approaches creates powerful specialized tools.

Developing custom features for blue sky scenes and centered objects underlined how domain knowledge significantly improves matching quality within targeted scenarios. The iterative design process - creating small test sets, adjusting weights, and evaluating results - provided practical experience in feature engineering methodology that extends beyond this specific assignment to general computer vision problem-solving.

Acknowledgements

This assignment was completed with reference to a variety of academic, technical and online resources.

Course Resources:

- Professor Bruce Maxwell for project specification, assignment structure, CSV utility functions, face detection implementation, sample histogram code, sample read file code.
- CS 5330 course lectures and materials for foundational concepts in feature extraction, distance metrics, and content-based image retrieval systems
- Piazza posts for some generic clarifications on assignment requirements and details

Technical Documentation:

- OpenCV documentation (https://docs.opencv.org/) for image processing functions, DFT implementation, Gabor kernel generation, and filter operations
- Gradio documentation (https://www.gradio.app/docs) for web interface development, component APIs, and event handling
- Python subprocess module documentation for process execution and output capture

Research and Theoretical Background:

- Varma, M., and Zisserman, A. "Texture Classification: Are Filter Banks Necessary?" *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2003. Referenced for Laws filter implementation and texture analysis approaches.

Implementation Resources:

- GeeksforGeeks tutorials on histogram intersection and cosine distance calculations
- Stack Overflow community for troubleshooting Gradio 6.0 compatibility issues and Windows-specific dirent.h implementation
- PIL/Pillow documentation for image loading and thumbnail generation in Python interface

Development Assistance:

- Claude AI (Anthropic) for analytical reasoning in distance metric design, weight tuning strategies for combined features, architectural design discussions for extensible code structure, and documentation support including paraphrasing, grammar review, and clarity improvements
- Debugging assistance for histogram normalization issues and feature vector size validation

Code Reuse:

- Assignment 1 filter implementations: Sobel filters (sobelX3x3, sobelY3x3, magnitude) and Gaussian blur (blur5x5_2) integrated via filter.h header from previous project
- Assignment 1 face detection: Haar Cascade implementation and detection functions reused for face-aware feature extraction